

# GENESIS MINER

## Relatório técnico completo

Motor de mineração · Histórico canónico · Event log de elegibilidade

Período: ~19-21 agosto 2026

Produção: VM 177.7.47.139 (srv1629706 / genesisdao.tech)

Frentes: 4B → 4C → 4D.1/4D.2 · Planos 2-5 · Auditorias RO · Deploy P3

*Documento consolidado: relatório all-time + anexos oficiais de cada entrega da sessão.*

## GENESIS MINER — RELATÓRIO TÉCNICO COMPLETO

### Refatoração do motor de mineração, histórico canónico e elegibilidade

Período: ~19-21 agosto 2026

Âmbito: current/ | Produção VM 177.7.47.139 (srv1629706 / genesisdao.tech)

Regra de ouro: não alterar fórmula económica de payout/yield sem decisão explícita.

Este PDF consolida o relatório all-time e os anexos oficiais de cada frente (4B/4C/4D.2, Planos 2-5), tal como entregues na sessão de engenharia.

# Relatório all-time — Refatoração do motor de mineração / histórico / elegibilidade

Período: ~19-21 ago 2026  
Âmbito: current/ | VM produção 177.7.47.139 (genesisdao.tech) | staging/local  
Regra transversal: não alterar a fórmula económica de payout/yield sem decisão explícita

## 1. Resumo executivo

Nesta linha de trabalho fechámos três frentes grandes e uma auditoria:

| Frente     | Objectivo                                                     | Estado                                    |
|------------|---------------------------------------------------------------|-------------------------------------------|
| 4C -> 4D.2 | Event log de elegibilidade + reconstrução pura + read adapter | CLOSED (prod)                             |
| Plano 3    | Histórico canónico 1 crédito / user x coin x janela           | CLOSED (prod)                             |
| Plano 4    | Auditoria de consistência (só leitura)                        | CLOSED                                    |
| Plano 5    | Blindagem: concorrência, atomicidade, mismatch, epsilon       | CLOSED (código + testes; sem deploy prod) |
| 5B / 6     | Resegmentar legado >10 min / replay físico                    | Não feitos                                |

Veredicto global: o motor actual é atómico, idempotente e concorrente-seguro sob o contrato de snapshot catch-up, com histórico canónico único em produção. O passado não é replay físico (limitação documentada, não “corrigida” à força).

## 2. Linha do tempo (o que foi feito)

### A. Elegibilidade (4C -> 4D.2) — CLOSED

- Tabela append-only mining\_eligibility\_events
  - Emissores em mutações instrumentadas (power, battery, coin, equip, check-in, etc.)
  - Cutover temporal (eventos válidos pós-cutover)
  - Core puro em mining-eligibility-reconstruction/
  - Read adapter 4D.2: empty@cutover + eventos [cutover, fromMs] -> initialState
  - Deploy na VM 139 (após tentativa na VM de teste 125)
  - Smoke: check-in + tráfego vivo a gravar eventos
- Intencionalmente fora: bootstrap pré-cutover (4D.3), ligação do reconstruction ao payout

### B. Plano 3 — Histórico canónico — CLOSED em produção

Problema: UI mostrava dezenas/centenas de “blocos” na mesma janela 10:40->10:50 (1 row por rack/ASIC).  
Correcção:

1. Writer (progress-computer.ts): consolidateMiningBlockHistoryRows antes do INSERT
  2. Economia intocada (totalGained / balances / yield / locks / ledger)
  3. Migration 20260821160000\_mining\_block\_history\_canonical: consolidar legado + UNIQUE (user\_id, coin\_id, window\_start\_ms, window\_end\_ms)
  4. UI: “Crédito da janela”; block\_time = parâmetro de rede
  5. API já 1:1 com rows
- Deploy prod (21 ago):

| Item              | Valor                                                               |
|-------------------|---------------------------------------------------------------------|
| Backup            | /root/minestation_backups/p3_history_20260821_151438/ (~82 MB dump) |
| Pré               | ~6,40 M rows   433 518 distinct   337 k chaves duplicadas           |
| Pós               | 433 518 rows   0 duplicidades   UNIQUE criada                       |
| Duração migration | ~11 s (após hotfixes oficiais no SQL)                               |
| App               | stop -> migrate -> rebuild -> healthy   PID novo                    |

Hotfixes na migration oficial (antes do COMMIT bem-sucedido): \$\$ em comentário; invariante float->epsilon; ownership da sequence.

**C. Plano 4 — Auditoria de consistência — CLOSED (só leitura)**

Pergunta-chave: catch-up = sequencial?

| Cenário                         | Resultado                                                           |
|---------------------------------|---------------------------------------------------------------------|
| Estado + yield fixos            | SIM (+/- float)                                                     |
| Estado muda a meio do intervalo | NÃO — snapshot actual pinta o intervalo todo (contrato, não bug P3) |

Achados principais:

- 0 duplicidades / 0 overlaps em prod (pós-P3)
- Gaps existem (muitos legítimos; sem replay não se prova “bug”)
- SUM history lifetime != coin\_balances não é invariante (histórico é espelho, não ledger)
- Eligibility log não alimenta o crédito actual (duas fontes potenciais no futuro)
- ~241 k rows legado com duração > 10 min
- Precisão float: delta ~1e-11-1e-14 -> aceitável hoje

**D. Plano 5 — Blindagem — CLOSED em código (sem mutação/deploy prod)**

| Entrega                        | Detalhe                                                              |
|--------------------------------|----------------------------------------------------------------------|
| Epsilon oficial                | MINING_AMOUNT_ABS_EPSILON=1e-9, REL=1e-12                            |
| Mismatch history               | mesma chave + amount incompatível -> ROLLBACK (sem saldo / sem last) |
| Contrato snapshot              | documentado no header do progress-computer                           |
| REQUIRE_REDIS_LOCK=1 sem Redis | fail-closed                                                          |
| Fault hooks                    | F1-F5 para testes de atomicidade                                     |
| Testes unitários               | 39/39 (P5 + regressão progress-computer)                             |
| Testes PG+Redis                | 4/4 (concorrência 2/5/10/20, retry, F1-F5, mismatch, Redis)          |

Produção P5:

DATABASE MUTATION: NÃO | PRODUCTION DEPLOY: NÃO

**3. Arquitectura resultante (estado actual)**

```

Mutação / poll
v
computeProgressForUser <- snapshot FÍSICO actual em [last, creditCap]
v
Redis lock (ou fail-closed se REQUIRE)
FOR UPDATE + last estável
ledger mp:uid:last:lastWrite
assert history compatível (+/- epsilon)
coin_balances += totalGained          <- ECONOMIA
mining_block_history 1x canónica      <- ESPELHO
last_updated_at = creditCap
COMMIT

```

Elegibilidade: event log paralelo para reconstrução futura; não decide o crédito hoje.

**4. Números-chave (produção, pós-P3)**

| Métrica                         | Valor                                      |
|---------------------------------|--------------------------------------------|
| History rows                    | ~434 k (cresce com ticks novos)            |
| Distinct canónico               | = rows (0 dups)                            |
| UNIQUE                          | mining_block_history_user_coin_window_uidx |
| Rows <= 10 min                  | ~194 k                                     |
| Rows > 10 min (legado)          | ~241 k                                     |
| Users com janelas longas        | ~925                                       |
| Eligibility events (amostra P4) | ~111 / 9 users (cresce com uso)            |

## 5. O que não foi feito (de propósito)

| Item                                  | Motivo                                  |
|---------------------------------------|-----------------------------------------|
| 4D.3 bootstrap pré-cutover            | Bloqueado / separado                    |
| Replay físico no crédito (Plano 6)    | Precisa decisão arquitectural           |
| Plano 5B resegmentação legado >10 min | Só auditado; backfill separado          |
| Migrar dinheiro para NUMERIC          | Separado (precisão financeira)          |
| Deploy do código P5 em prod           | Só implementado + testado local/staging |
| Alterar yield / payout / fórmula H    | Regra de ouro                           |

## 6. Ficheiros centrais tocados

Motor / histórico

- server/modules/mining-engine/services/progress-computer.ts
- mining-economic-epsilon.ts (P5)
- wall-clock-grid.ts, nft-room-mining.ts
- prisma/migrations/20260821160000\_mining\_block\_history\_canonical/

Elegibilidade

- mining-eligibility-events\*, mining-eligibility-cutover.ts
- mining-eligibility-reconstruction/
- mining-eligibility-read-adapter.ts
- migration 20260821120000\_mining\_eligibility\_events

UI

- CalculatorPage.tsx, PlayerCalculator.tsx

Testes

- progress-computer.test.ts, progress-p5-hardening.test.ts
- tests/integration/pg/mining-progress-p5.test.ts

## 7. Riscos residuais (abertos)

1. Catch-up usa snapshot actual (não H/ON/coin históricos por janela).
2. Legado >10 min ainda no histórico (chave única, grelha não uniforme).
3. Eligibility log != payout (seguro hoje; perigoso se ligarem sem bootstrap).
4. Float64 como tipo monetário (aceitável; não “ledger legal”).
5. Código P5 (mismatch abort) ainda não está no binário de produção até próximo deploy.

## 8. Recomendações seguintes

| Prioridade | Plano     | Acção                                                  |
|------------|-----------|--------------------------------------------------------|
| P0 ops     | Deploy P5 | rsync + rebuild na 139 (sem migration nova)            |
| P1 dados   | 5B        | Resegmentar legado >10 min com backup + invariante SUM |
| P2 produto | 6         | Decidir: manter snapshot vs replay via eligibility     |
| P3         | 4D.3      | Bootstrap pré-cutover só se for para replay            |

## 9. Nota lateral (suporte)

Relato de “não consigo pôr bateria no rack Dollar/NFT”: em geral regra de produto / dessync (armazém UUID, sala NFT, compatibleRacks, F5), não efeito do P3/P5 no crédito. Falta a mensagem exacta do alert para fechar.

## 10. Checklist de conformidade

[x] Economia de mining preservada

- [x] Histórico canônico em prod
- [x] Backup verificável no P3
- [x] Auditoria P4 sem mutação
- [x] Blindagem P5 com testes verdes
- [x] Zero backfill destrutivo improvisado
- [x] Eligibility reconstruction não ligada ao payout
- [ ] Código P5 em produção (pendente deploy)
- [ ] Resegmentação legado (5B)
- [ ] Replay físico (6)

---

Uma frase: passámos de histórico “1 linha por ASIC” e elegibilidade sem memória, para crédito canónico + event log + motor blindado sob contrato de snapshot, com auditoria formal e caminho claro para 5B/6 — sem ter mexido na economia real da mineração.

---

## TAREFA 4B — Spec canónica da identidade e da verdade histórica

Gates: nenhum ficheiro alterado | sem migration/schema/código | só evidência + contrato.

### 1. VEREDICTO EXECUTIVO

1. MiningUnit deve ser a unidade que contribui com hash para uma moeda num intervalo de tempo.
2. Identidade não é uniforme:
  - ASIC timed -> identidade = lease\_id (segue a lease; rack/slot são localização).
  - GPU normal / ASIC vitalícia -> itens de catálogo fungíveis; identidade económica = ocupação (user, rack\_id, slot\_index) + catalog\_item\_id enquanto equipada (não há UUID de instância no source).
3. O BD não permite  $\int \text{hash\_elegível}(t) | \text{yield}(t) dt$  correcto para o passado geral.
4. Modelo mínimo recomendado: híbrido — event log como fonte de verdade + projeção de intervalos para leitura.
5. repairEquippedAsicLeasesForRack é CRÍTICO: pode inventar acquired\_at/expires\_at a partir de now.
6. 4C deve persistir eventos + (opcional) soft-expire de leases; não ainda reescrever o cálculo do progress.

### 2. INVENTÁRIO DAS MINING UNITS

#### A. Máquina normal / GPU

| Campo                 | Valor (source)                                                                                                                         |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Onde nasce            | Shop / merge / gifts -> stock                                                                                                          |
| Persistido            | stock; equipado em rack_slots.machine_item_id                                                                                          |
| Identidade actual     | Catálogo item_id + posição no slot; sem instância UUID                                                                                 |
| Identidade candidata  | Enquanto equipada: (user_id, rack_id, slot_index) + catalog_item_id                                                                    |
| Expira?               | Não (sem lease timed)                                                                                                                  |
| Começa economicamente | Quando entra no slot e rack is_on+wiring+battery (gates no progress)                                                                   |
| Deixa de existir      | Unequip / troca de item no slot / rack off (gate)                                                                                      |
| Moeda                 | placed_racks.selected_coin_id (sala normal)                                                                                            |
| Hash                  | Soma base_production dos slots do rack x mult (listSlotMiningCredits sala normal = 1 crédito/rack)                                     |
| Nota                  | Em sala normal, várias GPUs no mesmo rack agregam num único crédito — identidade económica do progress = rackxcoin, não GPU individual |

Resposta à pergunta: GPU normal não tem identidade económica individual estável fora do slot; no crédito actual, o “átomo” em sala normal é o rack (slots somados), não cada GPU.

#### B. ASIC timed

| Campo      | Valor                                                                     |
|------------|---------------------------------------------------------------------------|
| Onde nasce | createAsicLeasesOnPurchase (shop, check-in reward, reconcile)             |
| Persistido | player_asic_leases + opcional rack_slots.machine_lease_id                 |
| Identidade | lease.id (UUID)                                                           |
| Expira     | expires_at; limpeza via expireUserAsicLeases (DELETE)                     |
| Começa     | acquired_at (compra); mineração só quando status=equipped + gates do rack |
| Moeda      | upgrades.nft_mining_coin_id (sala NFT)                                    |
| Hash       | 1 crédito/slot = base_production x mult daquele ASIC                      |
| Equip      | finalizeTimedMinerEquip só se isTimedAsicDuration                         |
| Unequip    | devolve lease a stock se ainda válida                                     |

#### C. ASIC vitalícia

| Campo       | Valor                                                                 |
|-------------|-----------------------------------------------------------------------|
| Diferença   | isTimedAsicDuration(cfg) === false -> sem linha em player_asic_leases |
| Onde existe | stock + rack_slots.machine_item_id; machine_lease_id tipicamente null |

|             |                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------|
| Identidade  | Como GPU: fungível; equipada = ocupação do slot + catalog id                                       |
| Equip path  | postApply não chama finalizeTimedMinerEquip se não timed (rack-aux-intent.controller.ts ~L508-509) |
| Moeda       | nft_mining_coin_id na sala NFT                                                                     |
| Move/remove | Unequip devolve qty a stock (não lease)                                                            |

## D. Multi-slot

| Facto                            | Evidência                                                                             |
|----------------------------------|---------------------------------------------------------------------------------------|
| Cada slot ASIC (NFT) = 1 crédito | listSlotMiningCredits L239-249                                                        |
| Dois ASICs iguais no mesmo rack  | Duas unidades económicas (dois slots / duas leases se timed)                          |
| Mudança de slot                  | Timed: lease mantém id, actualiza rack_id/slot_index; vitalícia/GPU: só muda ocupação |
| Identidade                       | Timed -> lease; vitalícia/GPU -> colocação no slot (não a “máquina física”)           |

## 3. IDENTIDADE CANÓNICA (proposta fechada)

```

MiningUnitIdentity =
  | { kind: 'lease',      userId, leaseId }
  | { kind: 'placement', userId, rackId, slotIndex, catalogItemId }
  | { kind: 'rack_aggregate', userId, rackId, catalogCoinContext } // sala normal actual

```

Justificação (source):

- Timed tem UUID real e move-se -> identidade = lease.
- Sem lease, stock é qty de item\_id -> não há “esta GPU #3”.
- Progress sala normal agrega slots -> até haver refactor, o atomo de crédito normal é rack, não slot; NFT é slot.

Regra: a identidade económica segue o que o motor já trata como crédito distinto; não inventar UUIDs fictícios para GPUs.

## 4. MATRIZ DE EVENTOS ECONÓMICOS

| Evento                | Onde                          | Eleg.           | Hash | Moeda | Loc/ID                  | Timestamp necessário? | Persistido hoje?                |
|-----------------------|-------------------------------|-----------------|------|-------|-------------------------|-----------------------|---------------------------------|
| Aquisição ASIC timed  | shop/checkin/createAsicLeases | sim (potencial) | —    | —     | cria lease              | sim (acquired_at)     | SIM (enquanto lease existir)    |
| Equip timed           | rack-aux + markLeaseEquipped  | sim             | sim  | —     | rack/slot               | sim                   | NÃO (só estado actual)          |
| Unequip timed         | rack-aux + release            | sim             | sim  | —     | limpa loc               | sim                   | NÃO                             |
| Equip GPU/vitalícia   | rack-aux                      | sim             | sim  | —     | slot+item               | sim                   | NÃO (só snapshot)               |
| Unequip GPU/vitalícia | rack-aux                      | sim             | sim  | —     | —                       | sim                   | NÃO                             |
| Expire + DELETE       | expireUserAsicLeases          | sim             | —    | —     | limpa slot              | sim (expires_at)      | timestamps perdidos após DELETE |
| Repair cria           | repairEquipped...             | risco           | —    | —     | cria id                 | inventa now           | SIM mas falso passado           |
| Reconcile stock       | reconcileTimedAsicStockLeases | stock           | —    | —     | cria/apaga stock leases | parcial               | operacional                     |
| is_on on/off          | save-servers                  | sim             | —    | —     | —                       | sim                   | NÃO (só valor actual)           |
| selected_coin         | save-servers / room-coins     | —               | —    | sim   | —                       | sim                   | NÃO                             |
| Wiring/battery        | rack-aux                      | sim (gate)      | —    | —     | —                       | sim                   | NÃO timeline                    |
| Multiplicador         | rack-aux                      | —               | sim  | —     | —                       | sim                   | NÃO timeline                    |
| Check-in freeze       | isCheckinFrozenForUser        | sim (zera)      | —    | —     | —                       | sim                   | PARCIAL (last_checkin_at_ms)    |
| Check-in bonus        | reward.ts += hps              | —               | sim  | —     | —                       | sim                   | PARCIAL (só total actual)       |
| Yield boundary        | yield-cron                    | —               | —    | —     | —                       | sim                   | SIM mining_yield_history        |

Separação:

- Económico: equip/unequip, power, coin, expire, freeze, bonus, hash config, yield.
- Operacional: sync stock, limpar slot após expire, repair de ponteiro machine\_lease\_id sem reinventar duração.

## 5. VERDADE ECONÓMICA VS ESTADO OPERACIONAL

| Caso                            | Estado operacional | Verdade económica           | Classificação                     |
|---------------------------------|--------------------|-----------------------------|-----------------------------------|
| Slot limpo por expire           | “não há máquina”   | Elegível até expires_at     | não reconstruível pós-DELETE      |
| Lease ainda no slot após expire | ainda “equipada”   | Deveria parar em expires_at | progress ignora lease -> inseguro |
| is_on=0 agora                   | off                | Off só desde última mudança | não reconstruível                 |
| Repair recia lease              | “consertado”       | Novo [now, now+duration]    | CRÍTICO — inventa vida            |

## 6. O QUE O BANCO CONSEGUE RECONSTRUIR

| Facto                                   | Reconstruível | Fonte                         | Precisão                            |
|-----------------------------------------|---------------|-------------------------------|-------------------------------------|
| Yield por boundary                      | Sim           | mining_yield_history          | EXACTA (após T1/T2)                 |
| ASIC timed ainda viva + sempre equipped | Parcial       | acquired_at/expires_at        | PARCIAL (sem gaps de unequip)       |
| Vitalícia/GPU sempre no mesmo slot      | Não           | só snapshot                   | IMPOSSÍVEL                          |
| Freeze se não houve novo check-in       | Parcial       | last_checkin_at_ms + política | PARCIAL                             |
| Bonus H/s no passado                    | Não           | só total actual               | IMPOSSÍVEL                          |
| Ledger de sync                          | Sim           | mining_progress_commit_ledger | que o batch ocorreu, não composição |

## 7. O QUE NÃO CONSEGUE RECONSTRUIR

- Instantes de equip/unequip/power/coin/wiring/battery/mult.
- Passado após DELETE de lease.
- Bonus check-in em cada instante.
- Freeze após check-ins posteriores (só guarda o último).
- Identidade no mining\_block\_history (sem rack/slot/lease).
- Distinguir duas vitalícias iguais excepto pelo slot actual.

## 8. EVENT LOG vs INTERVALOS vs HÍBRIDO

| Modelo       | Prós                                         | Contras                                                 |
|--------------|----------------------------------------------|---------------------------------------------------------|
| A Event log  | Completo, auditável, append-only             | Precisa projectar para query                            |
| B Intervalos | Leitura fácil para eligibleSegments          | Updates em unequip/expire mais frágeis; repair perigoso |
| C Híbrido    | Log = verdade; intervalos = cache de leitura | Mais peças; disciplina de projecção                     |

## 9. MODELO RECOMENDADO

Híbrido: Event Log (fonte de verdade) + intervalos materializados (opcional na 4C/4D).

Motivo: o source já é orientado a mutações (equip, save power, expire). Eventos batem com o domínio. Intervalos puros sem log não explicam \*porquê\* um segmento acabou e facilitam repair a reescrever história.

Mínimo para a pergunta canónica:

> Para qualquer  $t \in [\text{start}, \text{end})$ , quais units elegíveis, moeda e hash?

-> série de eventos com at\_ms + regras determinísticas de projecção (+ leases timed não apagadas).

## 10. FACTOS MÍNIMOS A PERSISTIR

| Facto                                                 | Estado                                                          |
|-------------------------------------------------------|-----------------------------------------------------------------|
| lease_id, acquired_at, expires_at                     | JÁ EXISTE — PRECISA SER PRESERVADO (não DELETE; status expired) |
| rack_slots.machine_item_id / machine_lease_id         | JÁ EXISTE (snapshot)                                            |
| placed_racks.is_on, selected_coin_id, wiring, battery | JÁ EXISTE (snapshot)                                            |
| Evento EQUIP / UNEQUIP com at_ms + identity           | PRECISA SER CRIADO                                              |
| POWER_CHANGED (is_on)                                 | PRECISA SER CRIADO                                              |
| COIN_CHANGED                                          | PRECISA SER CRIADO                                              |

|                                                |                                                         |
|------------------------------------------------|---------------------------------------------------------|
| WIRING / BATTERY / MULT changed (se gate/hash) | PRECISA SER CRIADO                                      |
| EXPIRED (marcar, não apagar)                   | PRECISA SER CRIADO (ou soft-delete)                     |
| CHECKIN / BONUS_DELTA                          | PRECISA SER CRIADO (ou aceitar bonus só snapshot na v1) |
| Identity no history (rack/slot/lease)          | PRECISA SER CRIADO (depois do motor)                    |

## 11. CONTRATO CONCEITUAL eligibleSegments(...)

eligibleSegments(identity, coinId, startMs, endMs, worldview) -> Segment[]

```
Segment = {
  startMs, endMs,          // half-open [start, end)
  coinId,
  hashContribution,        // H/s base (+ regra de bonus se incluída)
  identity,
  reasons?                 // debug: power, lease, freeze...
}
```

Dados mínimos: identidade; timeline de localização/equip; acquired\_at/expires\_at (timed); power; coin binding; freeze; (opcional v1) bonus; catalog hash/mult.

Quebra segmento: equip, unequip, expire, power, coin, freeze on/off, mudança de item/lease no slot, (mult/wiring/battery se afectam hash/gate).

Boundaries: eventos at\_ms u boundaries de yield 10 min u start/end do sync.

Semântica: [start, end) — evento em T afecta [T, ...); expire em expires\_at -> elegível em [..., expires\_at).

Intersectar:

syncInterval n lifetime(lease) n equippedIntervals n poweredOn n coinMatch n →frozen

depois x calculateIntegratedYield por pedaço.

## 12. AUDITORIA REPAIR / RECONCILE

### repairEquippedAsicLeasesForRack

| Pergunta           | Resposta                                            | Classe  |
|--------------------|-----------------------------------------------------|---------|
| Cria leases?       | Sim se slot timed sem lease e sem equipped match    | CRÍTICA |
| Apaga?             | Não directamente                                    | —       |
| Altera expires_at? | Novas leases: computeAsicLeaseExpiresAt(cfg, nowMs) | CRÍTICA |
| Muda rack/slot?    | Liga lease ao slot actual                           | RISCO   |
| Recria passado?    | acquired_at = nowMs — sim, inventa                  | CRÍTICA |

Re-ligar lease existente equipped ao slot = SEGURA (só ponteiro).

### reconcileTimedAsicStockLeases

| Cria/apaga leases stock | Sim                                                          |
|-------------------------|--------------------------------------------------------------|
| Classe                  | RISCO operacional (qty); não deve fingir equipped no passado |

## 13. CHECK-IN / FREEZE NO MODELO TEMPORAL

### Freeze

- Gate de elegibilidade no tick (racksRes = []) se frozen).
- Avança last\_updated\_at sem pagar.
- Evento temporal? Sim (estado em função de last\_checkin\_at\_ms + now).
- Reconstruível? Só enquanto o último check-in for o relevante; check-ins antigos não estão em série -> PARCIAL.
- Futuro: evento CHECKIN com at\_ms (e política) ou aceitar freeze só “desde último checkin conhecido”.

### Bonus

- checkin\_bonus\_hps acumulado em game\_states; += no reward.

- Aplicado no sync via effectiveHashWithCheckinBonus (não-NFT).
- Constante no intervalo? Assumido sim no snapshot; não há prova de série.
- Precisa evento? Para integral exacta: BONUS\_DELTA com at\_ms.
- Lacuna: v1 pode documentar bonus como snapshot-only; v2 com log de check-in.

## 14. DADOS LEGADOS E MIGRAÇÃO

| Risco                        | Impacto                                                                        |
|------------------------------|--------------------------------------------------------------------------------|
| Leases já DELETE             | Intervalos antigos irre recuperáveis                                           |
| History sem rack/slot/lease  | Recibos antigos sem identidade de unit                                         |
| Repair já reinventou expires | Durações económicas incorrectas possíveis                                      |
| Backfill completo            | IMPOSSÍVEL                                                                     |
| Coexistência                 | Novo modelo só para $t \geq \text{cutover}$ ; passado = best-effort / só yield |

Política: cutover timestamp; sem fingir precisão pré-cutover.

## 15. ESCOPO EXACTO DA FUTURA 4C

4C (persistência mínima) — implementar depois:

1. Soft-expire: status='expired' (ou equivalente); não DELETE imediato; limpar slot sem apagar factos.
2. Tabela/append-only mining eligibility events (nome a definir) com: identity, at\_ms, type, payload.
3. Emitir nos paths: equip/unequip miner, power/coin saves, expire, (opcional) wiring/battery/mult, checkin.
4. Endurecer repair: nunca criar lease com duração full a partir de now sem política explícita admin; preferir só reattach.
5. Não alterar ainda a fórmula de computeProgressForUser.
6. Não UNIQUE history nesta fase.

4D: projectar segmentos + trocar crédito para  $\int \text{hash\_elegível} | \text{yield}$ .

4E: identity no history / idempotência por unit.

## 16. CONFIRMAÇÃO

[x] Nenhum ficheiro foi alterado  
[x] Nenhuma migration / schema / tabela  
[x] Nenhum cálculo económico / progress / expire / UI alterado  
[x] Conclusões ligadas ao source (paths citados na 4A/4B)  
[x] Lacunas declaradas (IMPOSSÍVEL / PARCIAL / CRÍTICO)  
[x] MiningUnit + identidades normal / timed / vitalícia  
[x] Matriz de eventos  
[x] Modelo recomendado: HÍBRIDO (event log + projeções)  
[x] Lista exacta para 4C  
[x] Contrato eligibleSegments para 4D consumir sem redescobrir

Resumo numa frase: a verdade económica futura é um diário de mutações com timestamp + leases timed preservadas; o snapshot actual continua a ser só o presente operacional — nunca o substituto do passado.

## A. Ficheiros alterados

```
prisma/schema.prisma
-> model mining_eligibility_events + comentário status expired
-> schema append-only + índices

prisma/migrations/20260821120000_mining_eligibility_events/migration.sql
-> tabela + índices + UNIQUE parcial ASIC_EXPIRED(lease_id)
-> cutover físico da verdade histórica

server/modules/mining-engine/services/mining-eligibility-cutover.ts
-> CUTOVER_T (default UTC 2026-08-21 12:00) + env override

server/modules/mining-engine/services/mining-eligibility-events.ts
-> recordMiningEligibilityEvent (append-only)

server/modules/mining-engine/services/asic-lease.ts
-> soft-expire (sem DELETE no expire); eventos equip/unequip/expire; repair só reattach

server/modules/servers/controllers/rack-aux-intent.controller.ts
-> emit equip/unequip/aux (wiring/battery/mult) + remove-rack placements

server/modules/servers/controllers/save-servers.controller.ts
-> RACK_POWER_CHANGED / RACK_COIN_CHANGED só se valor mudou

server/modules/servers/controllers/room-coins.controller.ts
-> RACK_COIN_CHANGED (+ power se coin null força off)

server/modules/checkin/services/checkin.ts
-> CHECKIN_RECORDED no check-in bem-sucedido

tests/modules/mining-engine/services/mining-eligibility-events.test.ts
-> casos 1-8 (fake PoolClient) + contrato schema/cutover
```

## B. Migration

- Tabela mining\_eligibility\_events: id, user\_id, event\_type, at\_ms, identity\_kind, lease\_id, rack\_id, slot\_index, catalog\_item\_id, coin\_id, payload (jsonb), created\_at
- Índices: (user\_id, at\_ms), (lease\_id, at\_ms), (rack\_id, at\_ms)
- Unique parcial: um ASIC\_EXPIRED por lease\_id
- Leases: sem nova coluna; status = 'expired' no text existente (sem DELETE no expire)

## C. Eventos implementados

| Tipo                        | Quando                                        | Path                                         | Dados                                            |                                             |
|-----------------------------|-----------------------------------------------|----------------------------------------------|--------------------------------------------------|---------------------------------------------|
| MINER_EQUIPP<br>ED          | equip timed / placement                       | finalizeTimedMinerEquip /<br>equip postApply | lease ou placement +<br>rack/slot/item/at_ms     |                                             |
| MINER_UNEQUI<br>PPED        | unequip / expire de equipped /<br>remove rack | release* / expire / remove                   | identidade pré-limpeza                           |                                             |
| ASIC_EXPIRED                | transição stock\                              | equipped -> expired                          | expireUserAsicLeases /<br>soft-expire em release | lease + acquired/expires<br>preservados; 1x |
| RACK_POWER_<br>CHANGED      | is_on muda de verdade                         | save-servers, room-coins<br>(clear coin)     | rack + novo/prev                                 |                                             |
| RACK_COIN_CH<br>ANGED       | selected_coin_id muda (não<br>NFT)            | save-servers, room-coins                     | rack + coin                                      |                                             |
| RACK_WIRING_<br>CHANGED     | wiring muda                                   | aux equip/unequip                            | rack + prev/next                                 |                                             |
| RACK_BATTERY<br>_CHANGED    | battery muda                                  | aux                                          | rack + ids                                       |                                             |
| RACK_MULTIPLI<br>ER_CHANGED | slot mult muda                                | aux                                          | rack + slot + item                               |                                             |
| CHECKIN_RECO<br>RDED        | check-in commit                               | checkin service                              | user + streak/prev                               |                                             |

## D. Wiring / battery / multiplier / freeze / bonus

| Área              | Decisão                                | Justificativa                                                                     |
|-------------------|----------------------------------------|-----------------------------------------------------------------------------------|
| Wiring            | EVENTO IMPLEMENTADO                    | Gate em progress-computer: sem wiring_id não minera                               |
| Battery           | EVENTO IMPLEMENTADO                    | Idem (battery_id)                                                                 |
| Multiplier        | EVENTO IMPLEMENTADO                    | Altera hash via rackMultiplierFactor                                              |
| Freeze / check-in | EVENTO IMPLEMENTADO (CHECKIN_RECORDED) | last_checkin_at_ms sozinho perde histórico ao sobrescrever; evento mínimo para 4D |
| Bonus H/s         | ADIADO (Opção B)                       | Snapshot-only; sem BONUS_DELTA; política actual intacta                           |

Nota: progress-computer ainda pode forçar is\_on=0 sem evento — não alterado (fora de escopo 4C). Gap conhecido para 4D.

## E. Repair / reconcile

Repair (antes): slot timed sem lease -> INSERT lease com acquired\_at=now, expires\_at=now+duration.

Repair (agora): só reattach se existir lease equipped no rack/slot; senão log asic\_lease\_repair\_orphan\_slot e não cria lease.

Reconcile stock: continua a alinhar inventário (pode DELETE stock excess nunca equipped). Não recria leases equipped nem altera timestamps económicos conhecidos.

## F. Testes

```
npx vitest run tests/modules/mining-engine/services/mining-eligibility-events.test.ts --no-cache  
-> 11 passed
```

```
npm run typecheck -> ok  
npm run build -> ok  
prisma migrate deploy -> 20260821120000 aplicada (local)
```

## G. Diff económico

```
computeProgressForUser: INALTERADO  
fórmula de payout: INALTERADA  
mining_block_history: INALTERADO
```

Cutover: MINING\_ELIGIBILITY\_HISTORY\_CUTOVER\_MS\_DEFAULT = 2026-08-21T12:00:00.000Z (override via MINING\_ELIGIBILITY\_HISTORY\_CUTOVER\_MS). Sem backfill fictício.

## A. Ficheiros

```
server/modules/mining-engine/services/mining-eligibility-reconstruction/
├─ types.ts          -> estado, intervalos, diagnostics, API types
├─ order.ts          -> ORDER BY at_ms ASC, id ASC (+ normalize)
├─ evaluate.ts       -> evaluateMinerEligibility + clone
├─ reducer.ts        -> applyEligibilityEvent (puro)
├─ reconstruct.ts    -> reconstructEligibilityTimeline
└─ index.ts          -> barrel público

tests/.../mining-eligibility-reconstruction.test.ts
-> casos 1-10 + isolamento económico
```

Nenhum ficheiro de payout / progress / history alterado (excepto o core novo).

## B. State machine

Estado: racks (isOn, coin, wiring, battery, multipliers, requiresSelectedCoin) | miners (equipped, placement/lease, expiresAt) | lastCheckinAtMs

Eventos -> transições: equip/unequip/expire | power/coin/wiring/battery/multiplier | checkin

Reducer: next = applyEligibilityEvent(state, event) — sem I/O, sem relógio, sem IDs novos.

## C. API

```
reconstructEligibilityTimeline({
  initialState, // estado no início da janela (caller / futuro 4D.2)
  events,
  fromMs,
  toMs,         // exclusivo
  cutoverMs?,   // default: cutover 4C
  isCheckinFrozenAt?, // política injectável; omitida = freeze não bloqueia
  strict?,      // throw em inconsistências graves
}): { timeline, finalState, diagnostics }

evaluateMinerEligibility(state, miner, atMs, { checkinFrozen? })
```

Cobre [fromMs, toMs).

## D. Semântica temporal

| Regra              | Valor                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Ordem              | Ordena sempre por at_ms ASC, id ASC; se a entrada vier desordenada -> diagnostic OUT_OF_ORDER_INPUT |
| Intervalos         | [start, end)   duração > 0   merge de segmentos adjacentes iguais                                   |
| Mesmo at_ms        | Aplica todos os eventos (por id) antes de abrir o intervalo                                         |
| expires_at         | Fronteira temporal: elegibilidade acaba em expires_at, mesmo com ASIC_EXPIRED.at_ms posterior       |
| ASIC_EXPIRED.at_ms | Materializa equipped=false; não prolonga elegibilidade                                              |

## E. Gates (motor actual)

Do progress-computer / listSlotMiningCredits:

```
equipped
lease.expires_at não ultrapassado
rack conhecido
is_on
wiring presente
battery presente
selected_coin_id (se requiresSelectedCoin; salas NFT -> false no initialState)
CHECKIN_FROZEN — só se isCheckinFrozenAt for injectado
```

Não avaliados no 4D.1 (precisam catálogo): coin.is\_active, base\_production, moeda NFT por upgrade.

## F. Diagnostics

```
OUT_OF_ORDER_INPUT
INVALID_EVENT_PAYLOAD
UNKNOWN_LEASE
UNEQUIP_WITHOUT_EQUIP
DUPLICATE_EQUIP
UNKNOWN_RACK
IDENTITY_KIND_MISMATCH
UNKNOWN_EVENT_TYPE
EVENT_BEFORE_CUTOVER_IGNORED
```

---

## G. Testes

```
npx vitest run mining-eligibility
-> 29 passed (reconstruction 11 + events 18)

npm run typecheck -> ok
npm run build -> ok
```

---

## H. Diff económico

```
computeProgressForUser: INALTERADO
payout: INALTERADO
mining_block_history: INALTERADO
```

---

4D.1 entregue: núcleo puro de Eligibility Reconstruction. Próximo passo natural: 4D.2 (adapter de initialState / leitura de eventos da BD), ainda sem ligar ao payout.

---

4D.2 — CLOSED. Adapter read-only entrega initialState temporalmente honesto + eventos ordenados ao 4D.1. Typecheck e build passam; 13 testes do adapter + 11 do core OK.

## A. Ficheiros alterados

| Ficheiro                                          | Responsabilidade                                             |
|---------------------------------------------------|--------------------------------------------------------------|
| mining-eligibility-reconstruction/apply-events.ts | API pura applyEligibilityEvents (replay state-only)          |
| mining-eligibility-reconstruction/index.ts        | Exporta applyEligibilityEvents                               |
| mining-eligibility-reconstruction/types.ts        | Diagnostics do adapter na taxonomia                          |
| mining-eligibility-read-adapter.ts                | 4D.2: load SQL, bootstrap, normalização, probe de cobertura, |
| mining-eligibility-read-adapter.test.ts           | Casos 1-10 + isolamento económico                            |

## B. Estratégia de initialState

```
empty @ CUTOVER_T
├── [cutoverMs, fromMs) -> applyEligibilityEvents -> initialState @ fromMs
└── [fromMs, toMs)      -> eventos da janela -> 4D.1
```

- Nunca usa snapshot operacional actual como estado histórico.
- Base no cutover = emptyEligibilityState() (sem inventar MINER\_EQUIPPED).
- Incompletude: probe (default on) — slots equipados \*agora\* sem MINER\_EQUIPPED pós-cutover -> complete: false + PRE\_CUTOVER\_STATE\_UNKNOWN / INITIAL\_STATE\_INCOMPLETE. Também se bootstrap gera UNEQUIP\_WITHOUT\_EQUIP / UNKNOWN\_LEASE.

## C. Queries temporais

```
-- bootstrap e window (mesma forma)
WHERE user_id = $1 AND at_ms >= $2 AND at_ms < $3
ORDER BY at_ms ASC, id ASC
```

- Bootstrap: [cutoverMs, fromMs)
- Window: [fromMs, toMs) — evento em fromMs só na window; em toMs fora.

## D. API pública

```
loadEligibilityReconstructionInput(db, { userId, fromMs, toMs, cutoverMs?, probePreCutoverCoverage? })
-> { initialState, events, diagnostics, complete, cutoverMs, fromMs, toMs }

buildEligibilityInitialStateAt(db, { userId, atMs, cutoverMs?, probePreCutoverCoverage? })

reconstructEligibilityForUser(db, { ..., strict?, isCheckinFrozenAt? })
-> ReconstructionResult & { complete, loadDiagnostics }
```

## E. Diagnostics

Core (4D.1): OUT\_OF\_ORDER\_INPUT, INVALID\_EVENT\_PAYLOAD, UNKNOWN\_LEASE, UNEQUIP\_WITHOUT\_EQUIP, ...

Adapter (4D.2): INITIAL\_STATE\_INCOMPLETE, PRE\_CUTOVER\_STATE\_UNKNOWN, EVENT\_ROW\_NORMALIZATION\_FAILED, EVENT\_QUERY\_WINDOW\_INVALID

## F. Testes

13/13 pass: cutover empty, bootstrap equip+power-off, boundaries fromMs/toMs, id ASC, actual!=histórico, pré-cutover incompleto, payload inválido, expires\_at preservado, read-only mutations recusadas, convenience + isolamento.

## G. Diff económico

computeProgressForUser: INALTERADO  
payout: INALTERADO  
mining\_block\_history: INALTERADO

Sem projection persistida, sem backfill fictício, sem 4D.3.

---

4D.2-close audit — resultado decisivo.

Script read-only: scripts/audit-eligibility-cutover-coverage.mjs  
(node scripts/audit-eligibility-cutover-coverage.mjs)

Números (BD actual)

| Métrica                                 | Valor                                                 |
|-----------------------------------------|-------------------------------------------------------|
| Cutover                                 | 2026-08-21T12:00:00.000Z (~2.5h atrás neste ambiente) |
| Event log                               | 0 linhas                                              |
| Slots equipados agora                   | 138 220 (6 507 users)                                 |
| Cobertos por MINER_EQUIPPED pós-cutover | 0                                                     |
| incomplete_slot_ratio                   | 1.0                                                   |
| Racks is_on=1                           | 19 852                                                |
| Leases status=equipped                  | 25 159                                                |

Leitura

- 1. Dívida de cobertura não é residual — é o universo inteiro do estado operativo actual. Com empty @ cutover + zero eventos, qualquer loadEligibilityReconstructionInput para estes users sai complete: false.
- 2. Não vai “cair naturalmente” só com o tempo enquanto o event log estiver vazio. Mesmo depois de emitters a escrever, slots que nunca forem unequip/re-equip continuam gap permanente (Opção A).
- 3. Há um sinal operacional separado: tabela existe, mas ainda não há nenhum evento com ~2.5h pós-cutover e dezenas de milhares de racks activos. Antes de desenhar 4D.3, vale confirmar que a escrita 4C está a correr neste ambiente (deploy / path de emissão).

Implicação para A / B / C

| Opção                           | Face a estes números                                                                                         |
|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| A aceitar complete=false        | Honestidade OK; reconstrução completa quase inútil para histórico que inclui o parque actual.                |
| B bootstrap factual @ CUTOVER_T | Quase certamente necessária se a timeline pós-cutover tiver de cobrir o parque já                            |
| C híbrido                       | Provável: snapshot factual só no cutover + events depois; probe continua a marcar gaps se o snapshot falhar. |

Recomendação: não abrir 4D.3 de consumo/payout. Próximo passo de desenho = auditoria de emitters (porquê 0 eventos) + especificação do bootstrap factual (B/C) se o objectivo for janelas reconstruíveis sobre o parque vivo.

Estado:

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| 4C               | CLOSED                                                                     |
| 4D.1             | CLOSED                                                                     |
| 4D.2             | CLOSED                                                                     |
| 4D.2-close audit | DONE — cobertura = 0%; event log vazio                                     |
| 4D.3             | NOT STARTED — decisão A/B/C bloqueada até emitters + política de bootstrap |

## 4C / 4D.2 — Emitter Health Audit (read-only)

### A) VEREDITO

HEALTHY — neste ambiente, zero eventos explicam-se por ausência de mutations instrumentadas desde (e antes de) CUTOVER\_T.

Com ressalvas explícitas (não confundir com “emitters já provados em tráfego real”):

| Ressalva                                 | Evidência                                                                                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| RUNTIME GAP (blocker operacional futuro) | node dist/bootstrap/server.js PIDs desde 2026-08-19;<br>dist/.../mining-eligibility-events.js rebuild 2026-08-21 10:27 — processo não     |
| Ambiente local/dev                       | DATABASE_URL -> localhost:5432/minestation (minestation-local-db);<br>NODE_ENV=development; não há prova de que isto seja produção remota |

Resposta ao critério de aceite:

> Desde CUTOVER\_T, houve mutations instrumentadas que deveriam ter produzido eventos?

> Não (proxies de actividade: 0).

> Temos evidência de emitters saudáveis + só falta mutation?

> No código/testes/dist: sim. Em runtime vivo: ainda não exercitado; e o processo actual está stale relativamente ao dist de hoje.

### B) Matriz de emitters

| PATH                                           | EVENT(S)                                                     | CALLABLE (código)                                   | SAME TX                                                | RUNTIME EVIDENCE                                                                                             | STATUS                              |
|------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------|
| A rack-aux (equip/unequip/wiring/battery/mult) | MINER_*,<br>RACK_WIRING/BATTERY/<br>MULTIPLIER_*; expire via | Sim -> HTTP<br>controllers -><br>runRackAuxMutation | Sim (pool.connect +<br>BEGIN... emitter ...<br>COMMIT) | Sem<br>game_states.server_upd<br>ated_at >= cutover                                                          | Implementad<br>o; não<br>exercitado |
| B save-servers                                 | RACK_POWER_CHANGED<br>, RACK_COIN_CHANGED                    | Sim                                                 | Sim                                                    | Idem (também bump<br>server_updated_at)                                                                      | OK código; 0<br>uso                 |
| C room-coins                                   | RACK_COIN_CHANGED,<br>RACK_POWER_CHANGED                     | Sim                                                 | Sim                                                    | Idem                                                                                                         | OK código; 0<br>uso                 |
| D check-in                                     | CHECKIN_RECORDED                                             | Sim                                                 | Sim                                                    | last_checkin_at_ms >=<br>cutover = 0                                                                         | OK código; 0<br>uso                 |
| E expire                                       | MINER_UNEQUIPPED +<br>ASIC_EXPIRED                           | Sim (lazy em rack-aux /<br>release)                 | Sim                                                    | 2113 leases past-expire<br>s ainda equipped -> expire<br>não correu; 0 leases com<br>expires_at em [cutover, | OK código;<br>path não<br>disparado |
| F progress-computer<br>auto power-off          | RACK_POWER_CHANGED<br>via<br>recordRackPowerChange           | Sim                                                 | Sim (TX de crédito)                                    | max(last_updated_at) =<br>2026-08-19 (&lt;<br>cutover)                                                       | OK código; 0<br>uso<br>pós-cutover  |
| G bulk-batteries                               | battery/power delta                                          | Sim                                                 | Sim                                                    | bump server_updated_at<br>-> 0                                                                               | OK código; 0<br>uso                 |
| H equip/unequip                                | via asic-lease +<br>placement helpers                        | Sim                                                 | Sim (rack-aux TX)                                      | 0                                                                                                            | OK código; 0<br>uso                 |
| I remove-rack                                  | unequip/placement via<br>release paths no aux                | Parcial (via release<br>helpers)                    | Sim se via rack-aux                                    | 0                                                                                                            | OK nos paths<br>instrumentad        |
| J admin (recall-all /<br>delete-user)          | —                                                            | Não (dívida conhecida)                              | n/a                                                    | Sem evidência de admin<br>destrutivo pós-cutover                                                             | NON-BLOCKE<br>R para 0<br>eventos   |

recordMiningEligibilityEvent: INSERT real; skip só invalid / duplicate\_expired; não engole outros erros (propaga -> ROLLBACK da TX). Sem feature flag.

### C) Database evidence

| Item           | Valor                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------|
| DB fingerprint | minestation / public — mesmos 138 220 slots / 19 852 racks on / 25 159 leases<br>equipped / 0 events |

|                                          |                                                                                                                                   |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Tabela                                   | existe; colunas + índices + unique parcial ASIC_EXPIRED OK                                                                        |
| Migration                                | 20260821120000_mining_eligibility_events finished 2026-08-21T13:42:53Z (após CUTOVER 12:00Z)                                      |
| Eventos                                  | total 0; sem min/max at_ms                                                                                                        |
| CUTOVER efectivo                         | 2026-08-21T12:00:00.000Z (default; sem override env)                                                                              |
| game_states.server_updated_at >= cutover | 0 users                                                                                                                           |
| Check-ins >= cutover                     | 0                                                                                                                                 |
| Leases acquired_at >= cutover            | 0                                                                                                                                 |
| Última actividade proxy                  | max(server_updated_at) 2026-08-19T01:33Z (~61h atrás); max(last_updated_at) 2026-08-19T13:50Z; max(checkin) 2026-08-19; max(lease |
| pg_stat_activity                         | backends idle; último query_start 2026-08-19                                                                                      |

Conclusão DB: não houve mutation instrumentada pós-cutover nesta BD. O parque grande é estado anterior ao cutover, não prova de writes recentes.

D) Runtime evidence (sem secrets)

| Item                  | Valor                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| Processos             | node dist/bootstrap/server.js — PID ~800915 :3999, ~812038 :3987                                             |
| Start                 | 2026-08-19 ~14:35–14:43 (local)                                                                              |
| Dist emitters         | mtime 2026-08-21 10:27 — mais novo que o processo                                                            |
| cwd                   | workspace .../minestation/current                                                                            |
| Docker                | minestation-local-db (POSTGRES_DB=minestation); também existe genesis-spot-pg (não usado pelo .env auditado) |
| systemctl minestation | inactive                                                                                                     |
| Build em runtime      | não verificável além de: processo antigo + dist novo no disco                                                |

E) Causa dos zero eventos (demonstrável)

0 eventos <- 0 mutations instrumentadas desde CUTOVER  
<- última actividade de jogo ~2026-08-19 (pré-cutover)

Não demonstrado: INSERT a falhar, TX a fazer rollback silencioso, ou emitter chamado sem write.

Hipótese B (paths não exercitados) — confirmada.

Hipótese C (runtime antigo) — confirmada como risco, mas não necessária para explicar zeros (não houve tráfego).

A/D/E/F/G — sem evidência como causa dos zeros.

F) Gaps

BLOCKER (antes de confiar em tráfego real)

- 1. Reiniciar processos Node após o build que contém emitters (senão, RUNTIME GAP na primeira mutation).
- 2. Confirmar se a BD auditada é a de produção ou só snapshot local — não verificável além de localhost/dev.

NON-BLOCKER

- Admin recall-all / delete-user sem emitters (dívida 4C já conhecida).
- Expire lazy: 2113 leases overdue ainda equipped (só limpa quando rack-aux corre).

INCONCLUSIVO

- Saúde dos emitters sob carga real (ainda não houve mutation pós-cutover para observar).
- Se produção remota tem eventos noutra DATABASE\_URL.

G) Próxima ação (só proposta — não implementada)

- 1) Restart dos node servers deste workspace (fechar RUNTIME GAP).
- 2) Smoke read-only de observação: 1 mutation controlada em staging/local (equip OU check-in) e confirmar 1 linha em mining\_eligibility\_events – OU repetir a coverage audit após tráfego real.
- 3) Só depois: especificar bootstrap factual (4D.3 B/C).

Não abrir 4D.3 bootstrap enquanto o ponto (1)+(2) não fechar o RUNTIME GAP e não houver  $\geq 1$  evento real na BD alvo.

---

### **Inventário write-path (resumo estático)**

recordMiningEligibilityEvent -> INSERT INTO mining\_eligibility\_events ... RETURNING id; erros não-23505 propagam; sem catch que transforme falha em sucesso.

Contrato atomicidade: nos paths A-H acima, mutation + event usam o mesmo PoolClient dentro de BEGIN/COMMIT. Admin J: sem event client.

Testes existentes: mining-eligibility-events.test.ts 18/18 pass (harness isolado; não escreveu na BD de produção).

---

## Deploy verification — 4C + 4D.2

### 1. Processos / PIDs

|            | Antes                                             | Depois                       |
|------------|---------------------------------------------------|------------------------------|
| PIDs       | 800915 (:3999), 812038 (:3987) — desde 2026-08-19 | mortos                       |
| Novos      | —                                                 | 2199682 :3999, 2199675 :3987 |
| Start      | —                                                 | 2026-08-21 10:36:59          |
| Build dist | —                                                 | 2026-08-21 10:36:39          |

Processo depois do build. Health: GET :3999/api/health -> 200.

### 2. BD alvo

host: localhost:5432  
database: minestation  
user: postgres

Não é produção remota — é a BD local (minestation-local-db). Migration: sem pendências.

### 3. Mutation

POST http://127.0.0.1:3999/api/checkin  
user\_id: 19501  
auth: JWT access cookie (legacy sid desligado neste env)  
HTTP 200 — performed: true

### 4. Evento persistido

| Campo              | Valor                                    |
|--------------------|------------------------------------------|
| id                 | 1                                        |
| user_id            | 19501                                    |
| event_type         | CHECKIN_RECORDED                         |
| at_ms              | 1787323083498 (2026-08-21T14:38:03.498Z) |
| created_at         | 1787323083517                            |
| at_ms >= CUTOVER_T | true (cutover 2026-08-21T12:00:00.000Z)  |

Persistiu após COMMIT (visível em SELECT).

### 5. Testes

mining-eligibility-events / reconstruction / read-adapter: 42/42 pass  
npm run build: OK  
prisma migrate deploy: OK (0 pending)  
npm run typecheck: FALHA só em testes do adapter (Partial StoredEvent) — não no server build

### 6. Blockers restantes

| Item                               | Estado                                               |
|------------------------------------|------------------------------------------------------|
| Emitters no runtime local          | OK (smoke passou)                                    |
| Produção remota                    | não verificada — confirmar DATABASE_URL + restart lá |
| Bootstrap pré-cutover (138k slots) | fora de escopo -> 4D.3                               |
| typecheck nos testes               | nit de tipos no harness 4D.2 (não bloqueia runtime)  |

Carimbo local: build OK | migration OK | restart OK | >=1 evento real pós-cutover OK.

4C + 4D.2 estão prontas neste ambiente. Produção só após o mesmo smoke na BD/processo de produção.

## Auditoria — Histórico de Blocos (somente leitura)

### CAUSA PROVÁVEL

CAUSA PRINCIPAL: o motor grava uma linha de mining\_block\_history por crédito de mineração (rack normal ou slot NFT x moeda x janela de 10 min), não por “bloco de rede” nem por intervalo único do utilizador.

A UI mostra cada linha como “1 bloco(s) creditado(s)” porque credit\_blocks = 1 para cada janela cheia de 600 000 ms — rótulo enganador.

#### CAUSAS SECUNDÁRIAS / SINTOMAS

- Frontend: 1:1 com a API, sem agregação.
- block\_time: 600 no cartão é mining\_coins.block\_time (economia da rede), não a grelha de histórico — coincidência numérica com 10 min.
- Sem UNIQUE em (user, coin, room, window, ...) no histórico (só PK id).

### PROVA (fluxo)

```
HTTP/snapshot (servers/inventory)
-> computeProgressForUser (progress-computer.ts)
  -> listSlotMiningCredits (nft-room-mining.ts) // 1 crédito/rack OU 1/slot NFT
  -> gained += hash x yield // saldo agregado por coin_id
  -> buildMiningBlockHistoryRowsForCredit(...) // 1+ rows / crédito / janela 10min
  -> INSERT mining_block_history (N rows)
  -> UPSERT coin_balances (soma por coin, 1x)
-> snapshot.loadBlockHistoryRows (player-calculator/snapshot.ts)
-> CalculatorPage / PlayerCalculator: "N bloco(s) creditado(s)"
```

Trechos-chave:

```
* Materializa N linhas de mining_block_history (1 por janela canónica) para um
* crédito de slot/rackxcoin. O crédito económico do tick continua no intervalo
* completo – esta função só espelha o histórico.
...
const creditBlocks = dur === TEN_MIN_MS ? 1 : Math.max(1, Math.round(dur / TEN_MIN_MS));

for (const sc of slotCredits) {
  ...
  totalGained.set(sc.coinId, (totalGained.get(sc.coinId) || 0) + gained);
  blockHistoryRows.push(
    ...buildMiningBlockHistoryRowsForCredit({ ... effectiveHash, ... })
  );

  // sala normal: 1 crédito (soma slots) por rackxselected_coin
  // sala NFT: 1 crédito por slot ASIC x nft_mining_coin_id
```

### SEMÂNTICA ATUAL

| Termo                     | Significado real no código                                                                           |
|---------------------------|------------------------------------------------------------------------------------------------------|
| Tick / janela             | Grelha UTC 10 min (TEN_MIN_MS em wall-clock-grid.ts); creditCap = lastCompletedTenMinuteUtcGrid(now) |
| credit / gained           | effectiveHash x yield por crédito (rack ou slot), somado em totalGained -> coin_balances             |
| credit_blocks             | Nº de segmentos de 10 min nessa linha (1 se janela exacta); não = blocos de rede                     |
| block_time / block_reward | Metadados da moeda para yield/UI; não definem a grelha do histórico                                  |
| history row               | Espelho de um crédito x uma janela (hash + amount daquela contribuição)                              |
| block_id / tick_id        | Não existem — chave temporal = (window_start_ms, window_end_ms)                                      |

Unidade económica do payout: por coin\_id agregado no user, não por linha de histórico.

### CASO 10:40 -> 10:50 (prod 177.7.47.139)

Janela: window\_start=1787319600000 ... end=1787320200000 (BRT 10:40-10:50), POL block\_time=600,

block\_reward=0.55.

Exemplo alinhado ao teu padrão (user\_id=11915):

```
hps agregados: {7,6,3,2,2,2,1} -> 8 linhas, todas credit_blocks=1
racks ON com POL: 8 (room_initial=8, outra sala=1)
history room_initial: 7 rows + 1 outra sala = 8
```

Ou seja: várias contribuições (racks/slots) no mesmo intervalo, cada uma com o seu hashrate — não 8 “blocos de rede”.

User grande (7691): 67 linhas POL no mesmo window, sum(credit\_blocks)=67, sum(amount\_coins) = soma das contribuições (não 67x o mesmo payout).

## ECONOMIA

| Pergunta                                         | Veredicto                                                  |
|--------------------------------------------------|------------------------------------------------------------|
| Payout duplicado por linha de histórico?         | Não — saldo = SUM gained por coin_id numa única UPSERT     |
| Histórico multi-linha = multi-crédito económico? | Sim, por design (componentes); UI chama cada um de “bloco” |
| Duplicação visual?                               | Sim — N contribuições x “1 bloco(s)”                       |
| Duplicação de saldo pelo histórico?              | Não (histórico é espelho)                                  |

## IDEMPOTÊNCIA

- Saldo / tick: FOR UPDATE + re-check last\_updated\_at + ledger mining\_progress\_commit\_ledger (user\_id, idempotency\_key) UNIQUE com mp:{user}:{last}:{lastWrite}.
- Histórico: INSERT simples, sem ON CONFLICT / unique de janela. Se o ledger falhar/disabled e a corrida passar, o histórico pode ganhar linhas extra; o caminho feliz não reprocessa o mesmo [last, lastWrite] porque last\_updated\_at já avançou.

Resposta directa: \*o mesmo 10:40->10:50\* não é re-creditado no saldo no fluxo normal; nada impede semanticamente um segundo INSERT de history se o crédito correr outra vez com outro intervalo overlapping mal protegido — mas a causa do sintoma actual não é retry, é granularidade por crédito.

## FRONTEND

Duplicação não nasce no React por clone de rows.

- API: snapshot.ts mapeia cada row -> creditedBlocks: Math.max(1, credit\_blocks).
- UI: CalculatorPage.tsx / PlayerCalculator.tsx — um card por entry, texto “... bloco(s) creditado(s)”.

Origem: backend/persistência (granularidade) + copy da UI.

Classificação do problema observado: B (créditos legítimos por rack/slot) + apresentação (chamados de “blocos”). Não é D (API a duplicar a mesma linha).

## ARQUIVOS PARA O PLANO 2

1. server/modules/mining-engine/services/progress-computer.ts — quem gera N rows
2. server/modules/mining-engine/services/nft-room-mining.ts — granularidade rack vs slot
3. server/modules/mining-engine/services/wall-clock-grid.ts — janelas / credit\_blocks
4. server/modules/player-calculator/services/snapshot.ts — DTO / falta de agregação
5. client/src/features/calculator/ui/CalculatorPage.tsx
6. client/src/features/admin/ui/PlayerCalculator.tsx
7. (opcional) schema/DDDL mining\_block\_history + índices — hoje sem unique de janela

Fora de escopo mantido: mining-eligibility-reconstruction/, payout formula, computeProgressForUser (só se Plano 2 decidir mudar persistência/agregação — não neste audit).

## PLANO 2 — AUDITORIA DO MOTOR DE HISTÓRICO

BD alvo consultada: 177.7.47.139 -> container app -> postgres://postgres:\*@postgres\_app:5432/minestation (produção). Não é localhost. Sem mutações.

### 1. VEREDITO

| CAUSA PRINCIPAL            | Histórico grava 1 row por crédito de mineração x janela de 10 min, não 1 row por “bloco de rede” nem por userxcoinxjanela.          |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| CAUSAS SECUNDÁRIAS         | Sala NFT -> 1 crédito/slot; sala normal -> 1 crédito/rack; catch-up multiplica por N janelas; UI rotula cada row como “1 bloco(s)”. |
| Só apresentação            | Copy “bloco(s) creditado(s)” + ausência de agregação na API/UI.                                                                     |
| Problema real do motor     | Granularidade do espelho histórico (decomposição intencional). Não é double-pay no saldo.                                           |
| Agregação userxcoinxwindow | Preserva economia se SUM(amount_coins) / SUM(user_hash_hps) — o payout já soma assim em totalGained.                                |
| Writers                    | Único writer: computeProgressForUser -> INSERT INTO mining_block_history.                                                           |

### 2. WRITE PATH COMPLETO

Único writer encontrado (grep: só progress-computer.ts).

```
Callers:
  servers/services/state-snapshot.ts
  inventory/services/snapshot.ts
    v
computeProgressForUser(pool, userId, now)
    v
Redis lock: REDIS_LOCK_KEYS.miningProgressUser(userId)
    v
pool.connect() -> pg client
  (pré-cálculo FORA de BEGIN: racks, slots, listSlotMiningCredits, yield)
    v
BEGIN
  SELECT game_states FOR UPDATE
  re-check last_updated_at == last
  INSERT mining_progress_commit_ledger ON CONFLICT DO NOTHING
    v
  UPSERT coin_balances <- economia (totalGained por coin_id)
    v
  SAVEPOINT mining_block_history_sp
  INSERT mining_block_history <- N rows (unnest arrays)
  RELEASE / ROLLBACK TO se 42P01
    v
  UPDATE game_states.last_updated_at = lastWrite
COMMIT
```

| Pergunta                  | Resposta                                                                   |
|---------------------------|----------------------------------------------------------------------------|
| Função que escreve        | computeProgressForUser (INSERT directo SQL)                                |
| Builder                   | buildMiningBlockHistoryRowsForCredit                                       |
| Transaction               | Mesma TX do crédito (BEGIN...COMMIT no mesmo PoolClient)                   |
| Unidade de uma row        | 1 crédito (rack ou slot) x 1 segmento de grelha                            |
| Uma chamada -> N rows?    | Sim                                                                        |
| O que determina N         | #créditos elegíveis x #janelas em [last, lastWrite)                        |
| Loop rack?                | Sim                                                                        |
| Loop slot?                | Indirecto via listSlotMiningCredits (NFT: por slot; normal: slots somados) |
| Loop coin?                | Por crédito (sc.coinId)                                                    |
| Loop room?                | Via room_id do rack                                                        |
| Loop janela?              | Em listCreditHistoryWindows                                                |
| Outro caminho de escrita? | Não (admin/report só SELECT)                                               |

mining\_block\_history não está no schema.prisma — DDL legado / tabela física.

### 3. GRANULARIDADE ATUAL

| Writer                               | Unidade actual                        | Multiplicadores                                    | N rows? | Motivo                                 |
|--------------------------------------|---------------------------------------|----------------------------------------------------|---------|----------------------------------------|
| computeProgressForUser / sala normal | user x rack x selected_coin x window  | racks ON com wiring+battery; slots somados no rack | Sim     | 1 SlotMiningCredit por rack            |
| computeProgressForUser / sala NFT    | user x slot(ASIC) x nft_coin x window | 1 crédito por máquina NFT no slot                  | Sim     | loop por slot em listSlotMiningCredits |
| + catch-up longo                     | x #segmentos 10min                    | listCreditHistoryWindows(last, lastWrite)          | Sim     | 1 row por segmento por crédito         |

Gates que impedem crédito (e portanto history): is\_on, wiring, battery, check-in freeze, coin inactive (power-off), effectiveHashxyield <= 0, user blocked.

### 4. CASO REAL DA BD

Alvo: user 11915, POL coin\_1765301021089, janela BRT 10:40-10:50  
window\_start\_ms=1787319600000, window\_end\_ms=1787320200000

| id  | room         | hps      | coins                               | credit_blocks |
|-----|--------------|----------|-------------------------------------|---------------|
| ... | room_initial | 7.300848 | 0.0001854856                        | 1             |
| ... | room_initial | 6.222314 | 0.0001580843                        | 1             |
| ... | room_initial | 3.318567 | 0.0000843116                        | 1             |
| ... | room_initial | 1.866694 | x4 rows (3 initial + 1 room_177...) | 1             |
| ... | room_initial | 0.881494 | 0.0000223953                        | 1             |

Totais: 8 rows | sum\_hps=25.19 | sum\_coins=0.0006399779 | sum(credit\_blocks)=8

Racks POL ON: 9 (1 com 0 machines -> sem crédito) -> 8 racks com máquinas = 8 rows.

Prova: 1 row ~ 1 rackxcoinxwindow (sala normal).

### 5. ECONOMIA VS HISTÓRICO

```
contribuição (sc)
-> gained = effectiveHash x yield
-> totalGained[coinId] += gained    → UPSERT coin_balances (A)
-> buildMiningBlockHistoryRows...    → mining_block_history (B)
```

| Histórico é fonte do payout? | Não                                                               |
|------------------------------|-------------------------------------------------------------------|
| Espelho do payout?           | Sim (B) — montado no mesmo tick a partir das mesmas contribuições |
| History row altera saldo?    | Não — INSERT isolado; falha 42P01 não reverte o crédito           |

Invariante económico actual:

sum(amount\_coins) das rows do tick (por coin) ~ totalGained[coin] (módulo split por janelas no catch-up, cuja soma ainda equivale ao intervalo completo — coberto por testes).

### 6. IDEMPOTÊNCIA

#### Crédito económico

1. Redis lock por user
2. SELECT ... FOR UPDATE + last\_updated\_at imutável no compute
3. Ledger UNIQUE (user\_id, idempotency\_key) com mp:{user}:{last}:{lastWrite}
4. Avanço de last\_updated\_at -> próximo tick não reabre o mesmo intervalo

#### Histórico

- Sem UNIQUE de janela / crédito
- INSERT append-only na mesma TX
- Se TX faz rollback (ledger conflict / race), history também não commita

| Cenário                    | Saldo                           | History         |
|----------------------------|---------------------------------|-----------------|
| Sequencial mesmo intervalo | Skip (last já avançou / dtMs=0) | Sem novo INSERT |

|                            |                                                         |                                                             |
|----------------------------|---------------------------------------------------------|-------------------------------------------------------------|
| Retry após erro pré-COMMIT | Ledger/race -> skip ou reprocesa intervalo ainda aberto | Sem duplicar se COMMIT não ocorreu                          |
| 2 workers                  | Redis lock / ledger / race em last                      | Um ganha; outro skip — history não duplica no caminho feliz |

Se o mesmo intervalo for processado duas vezes com sucesso de saldo: o desenho normal impede. History não tem defesa própria se alguém contornar ledger+last (risco residual).

## 7. DATABASE / CONSTRAINTS

| PK                | id BIGSERIAL                                                                               |
|-------------------|--------------------------------------------------------------------------------------------|
| FK                | user_id -> users(id) ON DELETE CASCADE                                                     |
| UNIQUE além de PK | Nenhum                                                                                     |
| Índices           | (user_id, window_end), (user_id, coin_id, window_end), (coin_id, window_end), (window_end) |
| Prisma            | não modelado                                                                               |

## 8. JANELA TEMPORAL

| Tamanho             | TEN_MIN_MS = 600_000                                                                  |
|---------------------|---------------------------------------------------------------------------------------|
| Alinhamento         | Meia-noite UTC + nx10 min                                                             |
| creditCap           | lastCompletedTenMinuteUtcGrid(now)                                                    |
| last                | game_states.last_updated_at                                                           |
| Intervalo > 10 min  | Particionado em segmentos; catch-up capped 72h                                        |
| Parcial off-grid    | Primeira/última janela podem ser < 10 min; credit_blocks = max(1, round(dur/TEN_MIN)) |
| block_time da moeda | Independente (economia); POL=600s só coincidência                                     |

Mesma janela canônica -> múltiplas history rows por: rack, slot NFT, room (via rack), e (no catch-up) por cada segmento.

## 9. NORMAL VS NFT

|           | Normal                                   | NFT                                            |
|-----------|------------------------------------------|------------------------------------------------|
| Crédito   | 1 por rackxselected_coin (slots somados) | 1 por slotxnft_mining_coin_id                  |
| Economia  | Soma em totalGained[coin]                | Idem                                           |
| Histórico | 1 row/rack/janela                        | 1 row/ASIC/janela (ex.: 216 ASICs -> 216 rows) |

Agregação futura userxcoinxwindow pode somar ambos sem mudar payout, porque o saldo já ignora a decomposição.

## 10. API / UI

```
mining_block_history
-> snapshot.loadBlockHistoryRows (SELECT ... ORDER BY window_end DESC LIMIT)
-> blockHistoryByCoin[coin].push({ creditedBlocks: max(1, credit_blocks), ... })
-> CalculatorPage / PlayerCalculator
    "{creditedBlocks} bloco(s) creditado(s)"
```

- Sem GROUP BY / dedupe / agregação
- Ordenação: window\_end\_ms DESC
- Cada row = um card

## 11. DADOS REAIS (prod)

| Métrica                           | Valor     |
|-----------------------------------|-----------|
| COUNT(*)                          | 6 368 623 |
| Users                             | 992       |
| Coins                             | 15        |
| Janelas distintas (start,end)     | 108 671   |
| DISTINCT (user, coin, start, end) | 432 239   |

Excess vs userxcoinxwindow

5 936 384 (~93% das rows)

TOP multiplicidade: até 216 rows / (user,coin&gt;window) (NFT, 1 room, mesmo hps) — alinhado a 216 slots.

## 12. RISCOS

| Risco                                              | Severidade                          |
|----------------------------------------------------|-------------------------------------|
| UI interpreta contribuição como “bloco de rede”    | Alto (produto/confiança)            |
| Volume histórico (N créditos x janelas x catch-up) | Alto (storage/queries)              |
| Sem UNIQUE histórico                               | Médio (só se bypass de ledger/last) |
| Agregar na UI sem cegar room_id                    | Baixo se documentado                |

## 13. IDENTIDADE HISTÓRICA ACTUAL

row ~ (user\_id, coin\_id, room\_id, contribution\_hash~user\_hash\_hps, window\_start, window\_end)  
+ amount\_coins espelhando essa contribuição

Não há chave estável de “bloco”; id é só surrogado. Contribuições gêmeas (mesmo hps) -> rows distintas.

## 14. IDENTIDADE HISTÓRICA CANDIDATA

Baseada no motor (payout já agregado por coin):

(user\_id, coin\_id, window\_start\_ms, window\_end\_ms)

Com payload agregado:

- amount\_coins = SUM(contribuições)
- user\_hash\_hps = SUM(contribuições) (ou documentar média — SUM alinha ao poder total do tick)
- credit\_blocks = 1 se janela exacta de 10 min
- room\_id: omitir ou 'multi' / lista — não faz parte da chave se a unidade for userxcoinxwindow

## 15. ARQUIVOS QUE O PLANO SEGUINTE TERÁ DE ALTERAR

1. server/modules/mining-engine/services/progress-computer.ts
2. server/modules/mining-engine/services/nft-room-mining.ts (só se granularidade de crédito mudar — provavelmente não)
3. server/modules/mining-engine/services/wall-clock-grid.ts (só se mudar semântica de janela — provavelmente não)
4. server/modules/player-calculator/services/snapshot.ts
5. client/.../CalculatorPage.tsx + PlayerCalculator.tsx (copy/agregação)
6. (opcional) DDL UNIQUE / backfill — fora deste plano

## 16. O QUE NÃO DEVE SER ALTERADO

- Fórmula de gained / totalGained / coin\_balances
- computeProgressForUser como motor económico (só o espelho histórico, se mudar)
- mining-eligibility-reconstruction/
- Event log 4C
- Payout / yield rates / mining\_coins economics

### Invariantes a blindar no próximo plano

1. SUM(history.amount\_coins) por (user, coin, window) = contribuição económica daquele intervalo para essa moeda.
2. Reprocessar o mesmo [last, lastWrite] não credita saldo duas vezes.
3. Agregação não inventa yield; só colapsa decomposição rack/slot.
4. credit\_blocks / label UI não devem implicar mining\_coins.block\_time de rede.

Plano 3 implementado — o motor passa a produzir 1 crédito histórico por user x coin x janela; economia intacta.

### Writer

- consolidateMiningBlockHistoryRows em progress-computer.ts: SUM(amount/hash/usd), MAX(credit\_blocks), room\_id só se todas iguais
- INSERT só depois da consolidação; ON CONFLICT ... DO NOTHING quando a unique existir (fallback sem unique se a migration ainda não correu)

### Economia (intocado)

totalGained, balances, yield, locks, ledger, elegibilidade, payout

### BD

Migration 20260821160000\_mining\_block\_history\_canonical:

- consolida legado com GROUP BY canónico
- aborta se SUM(amount\_coins) diverge
- UNIQUE (user\_id, coin\_id, window\_start\_ms, window\_end\_ms)

### UI

CalculatorPage + PlayerCalculator: “Crédito da janela”; block\_time marcado como parâmetro da rede

### Testes

30/30 em progress-computer (incl. 8 racks -> 1 row; 216 ASICs x 30 min -> 3 rows). typecheck + build OK.

### API

snapshot.ts já devolve rows 1:1 — com writer + migration a unidade canónica chega sem agregação na UI.

Deploy: aplicar a migration antes (ou junto) do restart do app — em prod ~6M rows -> ~432k; janela com lock. Queres que eu faça o deploy na VM 139?

---

## Relatório — Plano 3 em produção (VM 139 / srv1629706)

### Pré-checks (só leitura) — GO

| # | Check                             | Resultado                                                         |
|---|-----------------------------------|-------------------------------------------------------------------|
| 1 | DATABASE_URL do app               | postgres://postgres:*@postgres_app:5432/minestation               |
| 2 | Postgres alvo                     | contentor postgres_app (Postgres 16), rede app_production_default |
| 3 | Histórico pré                     | 6 402 797 rows   433 518 distinct   337 238 chaves duplicadas     |
| 4 | Migration P3                      | não aplicada (0 em _prisma_migrations)                            |
| 5 | Disco                             | 60 G livres; tabela ~1,6 GB                                       |
| 6 | Outras migrations na mesma tabela | nenhuma                                                           |
| 7 | App com código P3                 | não (seria rebuild após migration)                                |
| 8 | Backup                            | feito antes da consolidação (app parado)                          |

### Backup

- Path: /root/minestation\_backups/p3\_history\_20260821\_151438/mining\_block\_history.dump
- Formato: pg\_dump -Fc | 82 MB | TOC listável (pg\_restore -l OK)
- Stats pré-stop: SUM(amount\_coins) ~ 11777.03834990576

### Janela segura

- docker stop app @ 15:14:59 UTC (writers off)
- App down até rebuild healthy @ 15:22:05 UTC

### Migration

Aplicada exclusivamente 20260821160000\_mining\_block\_history\_canonical via psql (não migrate deploy completo).

Tentativas anteriores reverteram (estado intacto): comentário com \$\$; invariante float; ownership da sequence. Correções na migration oficial, depois reaplicação.

| COMMIT         | 15:21:29 UTC                               |
|----------------|--------------------------------------------|
| Duração        | 11 s                                       |
| Registo Prisma | finished_at=2026-08-21 15:21:31            |
| UNIQUE         | mining_block_history_user_coin_window_uidx |

### Pós-migration

| Métrica           | Antes             | Depois                                                     |
|-------------------|-------------------|------------------------------------------------------------|
| Rows              | 6 402 797         | 433 518                                                    |
| Distinct canónico | 433 518           | 433 518 (= rows)                                           |
| Duplicidades      | 337 238           | 0                                                          |
| SUM(amount_coins) | 11777.03834990576 | 11777.038349896544 (delta float ~1e-11, dentro do epsilon) |

### Build / processos

|            | Antes        | Depois                                                         |
|------------|--------------|----------------------------------------------------------------|
| Node PID   | 2581110      | 2643804                                                        |
| Started    | 14:50:09 UTC | 15:22:05 UTC                                                   |
| P3 no dist | 0 matches    | 3 (consolidateMiningBlockHistoryRows)                          |
| Health     | —            | container healthy; https://genesisdao.tech/health/ready -> 200 |

### Smoke (user controlado 17628, 7 racks)

- Rewind controlado de last\_updated\_at -> catch-up 3 janelas x 3 moedas = 9 rows
- maxPerKey = 1 (não 3x7 racks)
- Retry: 2º computeProgress sem novo crédito / sem duplicar history
- coin\_balances delta = offlineMined esperado
- mining\_eligibility\_events delta=0 | mining\_yield\_history delta=0
- GET /api/checkin/status -> 200
- GET /api/calculator/me -> 200; history por moeda sem janelas duplicadas

## Caso legado (user 11915)

Todas as janelas pós-consolidação: 1 row por (coin, window) (incl. POL).

### Alertas

1. Migration precisou de 3 hotfixes oficiais antes do COMMIT (\$\$ em comentário, epsilon float, ALTER SEQUENCE OWNED BY). Nenhum SQL improvisado ad-hoc na BD.
2. Downtime do app ~7 min durante backup+migration+rebuild.
3. Smoke rebobinou last\_updated\_at do user 17628 e creditou o catch-up económico correspondente (documentado; não “ajustou” saldos à mão).
4. Migrations 20260821040000 / 20260821120000 continuam fora de \_prisma\_migrations (eligibility já existia por apply manual anterior) — não tocadas neste deploy.

Plano 3 em produção: concluído. Sem alterações adicionais após o smoke.

---

# PLANO 4 — AUDITORIA DE CONSISTÊNCIA DO MOTOR

Modo: somente leitura. Nenhuma mutação em produção, código ou config.

## 1. VEREDITO EXECUTIVO

Resposta à pergunta central:

> Para a mesma sequência de estado de mineração (hashrate, ON/OFF, moedas, yield fixos durante o intervalo), o resultado económico e o histórico por janela são determinísticos e associativos: catch-up T0->T6 = processamento sequencial T0->T1...->T6, dentro de ruído IEEE-754 (~1e-14-1e-11).

Mas o motor não reconstrói o estado físico ao longo do tempo. Catch-up aplica o snapshot actual (placed\_racks / slots / is\_on) a todo o intervalo [last, creditCap). Portanto:

| Cenário                                              | Catch-up = sequencial?                                                                 |
|------------------------------------------------------|----------------------------------------------------------------------------------------|
| Estado constante no intervalo + mesma curva de yield | SIM (prova local + testes)                                                             |
| Hashrate / ON-OFF / moeda mudam a meio do intervalo  | NÃO — by design; sequencial veria cada tick; catch-up “pinta” o passado com o presente |

Pós-P3: a unidade histórica canónica (user, coin, window) está única em produção (0 duplicidades). Isso não prova sozinho que saldo lifetime = SUM histórico lifetime.

Veredicto global: PASS CONDICIONAL — verdade económica por janela é determinística dado o snapshot; não há dual-payout por rack/ASIC na economia; há limites de desenho (sem time-travel, histórico legado longo, ON CONFLICT DO NOTHING, float) a tratar no Plano 5 se quiserem “replay físico”.

## 2. FLUXO ECONÔMICO REAL

```
callers: /api/servers/state , /api/inventory (throttle)
  v
computeProgressForUser(pool, uid, now)
  v
Redis lock (se efectivo) + creditCap = lastCompletedTenMin(now)
  v
READ (fora da TX): game_states.last, racks, slots, yield_history
  v
Para cada rack elegível (is_on ^ wiring ^ battery; !checkinFrozen):
  listSlotMiningCredits -> 1 crédito/rack (sala normal) ou 1/ASIC (NFT)
  gained = effectiveHash x integrateYield(last -> last+dt)
  totalGained[coin] += gained          <- ECONOMIA
  blockHistoryRows += split(janelas)  <- ESPELHO
  v
BEGIN
  FOR UPDATE game_states -> race se last mudou
  INSERT ledger ON CONFLICT DO NOTHING (chave mp:uid:last:lastWrite)
  UPSERT coin_balances += totalGained
  INSERT mining_block_history consolidado (UNIQUE; ON CONFLICT DO NOTHING)
  UPDATE last_updated_at = lastWrite
COMMIT
```

Economia != histórico como fonte: o saldo nunca é derivado do histórico. O histórico é espelho consolidado do mesmo gained.

## 3. FLUXO TEMPORAL REAL

| Conceito                 | Semântica                                                                          |
|--------------------------|------------------------------------------------------------------------------------|
| TEN_MIN_MS               | 600 000 ms, grelha UTC meia-noite                                                  |
| creditCap                | último boundary <= now (janela aberta não paga)                                    |
| last -> lastWrite        | um intervalo contínuo por tick (dt capped em MAX_EARNING_WINDOW_MS ~ 3 dias)       |
| listCreditHistoryWindows | parte [start,end) em segmentos (10 min exactos se alinhados; parciais se off-grid) |
| Writer novo              | 1 row canónica / janela após consolidateMiningBlockHistoryRows                     |

Produção (pós-P3): ainda existem ~241 k rows com duração > 10 min (legado pré-segmentação / catch-up antigo). O writer actual já não cria agregados T0->T6 numa única row.

## 4. INVARIANTES ENCONTRADOS

### Tolerância decimal (definição explícita)

| Camada                              | Tipo                                                                     | Notas                                       |  |  |
|-------------------------------------|--------------------------------------------------------------------------|---------------------------------------------|--|--|
| JS                                  | number (IEEE-754 binary64)                                               | toda a matemática                           |  |  |
| mining_block_history.amount_*       | double precision                                                         |                                             |  |  |
| coin_balances.amount                | double precision (cast via ::numeric[] no UPSERT)                        |                                             |  |  |
| Arredondamento                      | nenhum explícito no motor                                                | só Math.floor em timestamps / credit_blocks |  |  |
| Comparação de auditoria recomendada | $\text{abs}(a-b) \leq 1e-9$ absoluto ou $\text{abs}(a-b)/\max(1, \dots)$ | a ) $\leq 1e-12$                            |  |  |
| P3 migrate                          | epsilon 1e-6 no SUM float                                                | delta observado ~1e-11                      |  |  |

Invariante por tick (estado fixo):

```
SUM history.amount_coins (janelas do tick, após consolidate)
= totalGained[coin] (+/- float)
```

Invariante lifetime SUM history = coin\_balances: não é válido como regra global (outras fontes de saldo; mining pré-histórico; gaps legítimos). Evidência: user 11915 hist~0.55 vs bal~14.05.

## 5. TESTE CATCH-UP VS SEQUENCIAL

### Prova (estado + yield fixos)

- calculateIntegratedYield(T0,T6) == SUM calculateIntegratedYield(Ti,Ti+1) -> delta 0 (caso de teste local).
- buildMiningBlockHistoryRowsForCredit(T0->T6) vs 6 ticks de 10 min -> maxAbs per-window = 0.
- Partições 1x60 / 2x30 / 3x20 / 6x10 -> SUM económico delta <= ~1e-14.
- Testes unitários já cobrem associatividade de yield e SUM(janelas)~full (toBeCloseTo(..., 10)).

### Limite de desenho (FAIL se interpretado como “replay físico”)

Catch-up não aplica H distinto por janela a partir de eventos. Usa H actual em todas as janelas do intervalo.

```
Esperado “ideal”: 10:00 H=2, 10:10 H=6, 10:20 H=1
Catch-up actual: 10:00..10:30 -> H_ agora em todas
```

## 6. IDEMPOTÊNCIA

Quatro camadas (coerentes em conjunto):

| Camada                                             | Papel                                                                     |
|----------------------------------------------------|---------------------------------------------------------------------------|
| Redis lock                                         | evita dois workers no mesmo user (se Redis efectivo / REQUIRE_REDIS_LOCK) |
| FOR UPDATE + lastConfirmed == last                 | serializa; segundo aborta se last mudou                                   |
| mining_progress_commit_ledger                      | chave mp:uid:floor(last):floor(lastWrite) única                           |
| UNIQUE (user,coin>window) + ON CONFLICT DO NOTHING | última barreira no espelho                                                |

PASS para retry sem avanço de intervalo (ledger / last).

Risco residual: se existir row histórica antiga para a mesma janela com outro amount, DO NOTHING não corrige o espelho, mas o saldo ainda soma — divergência history↔economia possível em cenários patológicos (não observado no smoke P3).

## 7. CONCORRÊNCIA

Análise (sem corrida destrutiva em prod):

| Pergunta                     | Resposta                                                     |
|------------------------------|--------------------------------------------------------------|
| Quem ganha?                  | Quem obtém lock Redis e FOR UPDATE primeiro e passa o ledger |
| Quem perde?                  | Segundo: lock falha / last mudou / ledger DO NOTHING -> skip |
| Saldo 2x?                    | Não, se ledger + last intactos                               |
| History 2x?                  | UNIQUE impede; DO NOTHING                                    |
| last 2x?                     | Não na mesma chave de intervalo                              |
| Ledger identifica intervalo? | Sim: last+lastWrite                                          |

PASS (análise). Teste de stress concorrente real -> Plano 5 / staging.

## 8. ATOMICIDADE TRANSACIONAL

Ordem dentro de BEGIN...COMMIT:

| Falha                           | Saldo    | Histórico | last     |
|---------------------------------|----------|-----------|----------|
| Antes do ledger                 | 0        | 0         | 0        |
| Depois do ledger, antes balance | 0*       | 0         | 0        |
| Depois balance, antes history   | 0*       | 0         | 0        |
| Depois history, antes last      | 0*       | 0         | 0        |
| Antes COMMIT (qualquer)         | 0        | 0         | 0        |
| Após COMMIT                     | aplicado | aplicado  | avanzado |

\\*Rollback da TX desfaz ledger/balance/history/last.

Excepções intencionais (SAVEPOINT):

- Accrual account-manager: falha não reverte crédito.
- History 42P01: histórico desactivado; crédito + last avançam (espelho opcional).

PASS para o núcleo saldo/history/last quando a tabela history existe.

FAIL parcial se history ausente: economia avança sem espelho.

## 9. HASHRATE / RACK / SLOT / NFT

| Modo        | Créditos internos                             | Economia    | History pós-P3                |
|-------------|-----------------------------------------------|-------------|-------------------------------|
| Sala normal | 1 crédito = SUM slots do rack x selected_coin | totalGained | 1 row/janela/coin             |
| NFT         | 1 crédito / ASIC x nft_mining_coin_id         | idem        | 1 row/janela/coin (SUM ASICs) |

Quantidade de racks/ASICs não multiplica payout: só multiplica contribuições internas antes do consolidate. Testes P3: 8 racks -> 1 row; 216 ASICs x 3 janelas -> 3 rows.

PASS (decomposição != unidade económica).

Hashrate por janela no tempo: FAIL vs modelo "H histórico" (ver §5).

## 10. ON/OFF / WIRING / BATTERY

Elegibilidade no credit path:

```
is_on === 1 ^ wiring_id ^ battery_id
(+ moeda activa; senão force is_on=0)
+ !checkinFrozen
```

Sem leakage entre ticks sequenciais se o estado mudar entre ticks.

Com catch-up: OFF->ON no meio não é honrado por sub-janela — o snapshot no momento do compute decide o intervalo todo.

PASS para regra pontual; FAIL para boundaries temporais sob catch-up.

## 11. COIN / ROOM ISOLATION

- Coin: selected\_coin (normal) vs nft\_mining\_coin\_id (NFT); moedas nft\_room\_only bloqueadas fora da sala NFT.

- Room: não entra na chave canónica; room\_id consolidado = único se todas iguais, senão null/vazio (“múltiplas salas”).
  - Prod: ~105 k rows com room vazio; ~330 k com room set.
- PASS isolation de coin no writer.
- PASS room como payload, não identidade.

## 12. GAPS E OVERLAPS

| Check (prod RO)                    | Resultado |
|------------------------------------|-----------|
| Duplicidades canónicas             | 0         |
| Overlaps user+coin                 | 0         |
| start >= end                       | 0         |
| Gaps adjacentes (next_start > end) | 135 392   |
| Gaps > 10 min                      | 45 358    |

Gaps esperados quando: utilizador offline, freeze check-in, rack OFF, moeda inactiva, creditCap (janela aberta), cap de 3 dias, ausência de tick.

Não é possível, só com SQL, provar “gap = bug” vs “gap legítimo” sem reconstruir elegibilidade temporal (que o credit path não usa).

PASS no-overlap / no-dupe.

INCONCLUSIVO->FAIL metodológico para “sem gaps indevidos” sem replay de estado.

## 13. HISTÓRICO VS SALDO

| Afirmção                                    | Status                                             |
|---------------------------------------------|----------------------------------------------------|
| Por tick: SUM history do tick = totalGained | PASS (código + testes + smoke P3)                  |
| Lifetime SUM history = coin_balances        | FAIL como invariante (não foi desenhado assim)     |
| Amostra 11838                               | hist~0.925 vs bal~0.925 (quase; ruído/outras vias) |
| 11915                                       | hist<< balance                                     |

## 14. PRECISÃO NUMÉRICA

JS number -> x -> SUM map -> unnest float8 -> PG float8

- Associatividade de SUM float: delta ~1e-11 em milhões de rows (P3).
- Partições locais: delta ~1e-14-1e-15.

Classificação: aceitável para magnitudes actuais de payout (~1e-8-1e1 coins).

Risco económico real: baixo hoje; médio se no futuro usarem estes floats como ledger financeiro/auditoria legal -> preferir NUMERIC no Plano 5.

## 15. ELIGIBILITY / CUTOVER

| Facto                  | Evidência                             |
|------------------------|---------------------------------------|
| Event log existe       | 111 events / 9 users (prod)           |
| Cutover default        | 2026-08-21T12:00:00.000Z              |
| computeProgressForUser | não chama reconstruction/read-adapter |
| Credit path            | snapshot vivo placed_racks            |

Duas fontes potenciais: event-log (futuro/reconstrução) vs snapshot vivo (payout). Hoje não conflitam porque só o snapshot paga.

PASS “sem double source no payout actual”.

FAIL “eligibility -> progress integrado e temporalmente honesto”.

## 16. API / UI

- API GET /api/calculator/me: 1 row DB -> 1 entry blockHistory (sem re-agregar).
- UI: “Crédito da janela”; block\_time = parâmetro de rede, não duração do crédito.
- creditedBlocks ainda no DTO; UI não o usa como título principal.

PASS alinhamento canónico pós-P3.

Nota: credit\_blocks legado >1 ainda aparece em rows longas.

## 17. MATRIZ PASS/FAIL

| Invariante                          | Resultado             | Evidência                                  |
|-------------------------------------|-----------------------|--------------------------------------------|
| 1 row/user/coin/window              | PASS                  | SQL prod: 434 504 = distinct; UNIQUE index |
| Sem overlap                         | PASS                  | SQL: 0                                     |
| Sem gaps indevidos                  | INCONCLUSIVO          | gaps existem; falta replay de estado       |
| Catch-up = sequencial (estado fixo) | PASS                  | testes + prova local delta=0               |
| Catch-up = sequencial (estado muda) | FAIL (design)         | snapshot actual no intervalo               |
| Idempotência                        | PASS                  | código + testes ledger/race                |
| Concorrência                        | PASS (análise)        | lock+FOR UPDATE+ledger+UNIQUE              |
| Saldo tick = histórico tick         | PASS                  | código/testes/smoke                        |
| Saldo lifetime = histórico          | FAIL (não-invariante) | SQL amostra                                |
| Hashrate por janela (tempo)         | FAIL (design)         | sem time-travel                            |
| ON/OFF boundaries (catch-up)        | FAIL (design)         | idem                                       |
| Coin isolation                      | PASS                  | nft-room-mining + writer                   |
| Room isolation                      | PASS                  | room fora da chave                         |
| NFT vs normal                       | PASS                  | consolidate SUM                            |
| Transaction atomicity (core)        | PASS                  | TX única                                   |
| History opcional / accrual          | PASS parcial          | savepoints                                 |
| Determinismo (mesma entrada)        | PASS                  | sem dependência de ordem após SUM/Map      |
| Precisão numérica                   | PASS (aceitável)      | float classificado                         |
| Eligibility integration             | PASS parcial          | não no payout                              |
| API/UI canónico                     | PASS                  | snapshot + UI                              |
| credit_blocks != payout             | PASS                  | não entra na fórmula                       |
| Janelas todas 10 min                | FAIL legado           | 241 k rows longas                          |

## 18. RISCOS ENCONTRADOS

1. Catch-up “pinta” o passado com estado actual (H/ON/coin).
2. Histórico legado longo (credit\_blocks>>1, durações até 3 dias) — canónico por chave, mas não por grelha 10 min.
3. ON CONFLICT DO NOTHING pode preservar amount antigo se a janela já existir.
4. Eligibility log != motor de crédito — risco futuro se ligarem reconstruction ao payout sem cutover/bootstrap completo.
5. Float como tipo monetário.
6. Gaps não auditáveis sem estado temporal.
7. History table ausente -> crédito sem espelho (degradação consciente).

## 19. ARQUIVOS QUE REALMENTE PRECISARIAM SER ALTERADOS (Plano 5 — se houver)

Só se quiserem fechar os FAILs de desenho; Plano 4 não altera:

| Objectivo                  | Onde                                                                               |
|----------------------------|------------------------------------------------------------------------------------|
| Time-travel H/ON/coin      | progress-computer.ts + integração mining-eligibility-read-adapter / reconstruction |
| Re-segmentar legado longo  | migration/backfill controlado de mining_block_history                              |
| Espelho vs UNIQUE mismatch | política ON CONFLICT (UPDATE vs fail)                                              |
| Dinheiro exacto            | tipos NUMERIC + aritmética alinhada                                                |

Clarificar credit\_blocks

rename/docs/API; UI já parcialmente alinhada

Não precisam de mudança só para “PASS” do P3 canónico actual.

## 20. RECOMENDAÇÃO PARA O PLANO 5

Prioridade sugerida:

1. Decisão de produto: o motor deve ser “snapshot catch-up” (actual) ou “replay físico por janela”?
  - Se snapshot: documentar como contrato e fechar Plano 5 só com testes de regressão + stress concorrente em staging.
  - Se replay: Plano 5 grande — ligar eligibility pós-cutover ao crédito sem double-count.
2. Staging: teste concorrente real A/B no mesmo user.
3. Opcional: backfill de rows longas -> grelha 10 min (preservando SUM amount\_coins).
4. Opcional: NUMERIC + tolerância única no CI.
5. Não reabrir P3 de consolidação — canonicidade de chave está PASS.

### Critério de encerramento do Plano 4

Provamos que:

- o motor não duplica economia por rack/ASIC/room;
- produz uma verdade histórica canónica por (user, coin, window) em produção;
- com estado e yield fixos, catch-up e sequencial coincidem por janela e no saldo;
- não produz verdade temporal física sob mudanças de estado a meio do intervalo — isso é limitação de desenho, não bug do P3.

Plano 4 encerrado. Nenhuma alteração aplicada.

## PLANO 5 — RELATÓRIO FINAL

Blindagem concluída sem mudar fórmula económica, yield, payout, eligibility ou produção.

### A. Código alterado

| Arquivo                                                          | Função / peça                                                     | Motivo                                                       |
|------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| server/modules/mining-engine/services/mining-economic-epsilon.ts | amountsAlmostEqual, ABS/REL                                       | Epsilon oficial único                                        |
| progress-computer.ts                                             | header + contrato snapshot                                        | Contrato catch-up explícito; REQUIRE_REDIS_LOCK fail-closed  |
| progress-computer.ts                                             | MiningBlockHistoryMismatchError, assertCanonicalHistoryCompatible | Mesma chave + amount incompatível -> abort TX                |
| progress-computer.ts                                             | assertTickHistoryMatchesEconomy                                   | Invariante tick history ↔ totalGained                        |
| progress-computer.ts                                             | faultHooks em ComputeProgressOptions                              | Fault injection staging                                      |
| progress-computer.ts                                             | ordem TX: ledger -> assert history -> balances -> insert          | Mismatch antes do saldo efectivo persistir                   |
| tests/.../progress-p5-hardening.test.ts                          | unitários P5                                                      | Partições, snapshot, NFT/racks, mismatch, gaps               |
| tests/integration/pg/mining-progress-p5.test.ts                  | PG+Redis                                                          | Concorrência 2-20, retry, F1-F5, mismatch, Redis fail-closed |

Não alterado: totalGained fórmula, yield, wall-clock grid, eligibility/reconstruction, UI (já canónica no P3).

### B. Testes executados

#### Unitário (vitest)

39/39 PASS

- progress-p5-hardening.test.ts — 9
- progress-computer.test.ts — 30 (regressão)

#### PG integration (npm run test:pg-integration)

4/4 PASS

- concorrência 2/5/10/20 workers
- REQUIRE\_REDIS\_LOCK=1 sem Redis -> fail-closed
- faultHooks F1-F5 + retry
- mismatch canónico

typecheck PASS.

### C. Stress (amostra)

| workers | economic credit | ledger | history (user) n=distinct | last avanços   |
|---------|-----------------|--------|---------------------------|----------------|
| 2       | 1               | 1      | 1xdistinct                | 1 -> creditCap |
| 5       | 1               | 1      | idem                      | 1              |
| 10      | 1               | 1      | idem                      | 1              |
| 20      | 1               | 1      | idem                      | 1              |

Perdedores: race evitada (last mudou) / lock Redis. Sem double payout.

### D. Falhas injetadas

| Ponto            | Rollback esperado | Observado | Retry     |
|------------------|-------------------|-----------|-----------|
| F1 afterLedger   | tudo 0, last=T0   | PASS      | 1 crédito |
| F2 afterBalances | idem              | PASS      | 1 crédito |
| F3 afterHistory  | idem              | PASS      | 1 crédito |

|                     |      |      |           |
|---------------------|------|------|-----------|
| F4 afterLastUpdated | idem | PASS | 1 crédito |
| F5 beforeCommit     | idem | PASS | 1 crédito |

## E. Auditoria legacy (prod RO — sem mutação)

| Bucket            | Rows       |
|-------------------|------------|
| <= 10 min         | 194 411    |
| > 10 min          | 240 953    |
| > 30 min          | 200 584    |
| > 1 h             | 172 170    |
| > 3 h             | 121 136    |
| > 24 h            | 6 889      |
| credit_blocks > 1 | 238 309    |
| SUM(amount) >10m  | ~ 11533.06 |
| users com >10m    | 925        |
| coins             | 15         |

-> Base numérica pronta para Plano 5B (resegmentação), não executada aqui.

## F. Invariantes

| Invariant                      | Expected             | Observed    | Status |
|--------------------------------|----------------------|-------------|--------|
| Concorrência 2/5/10/20         | 1 credit             | 1           | PASS   |
| Retry idempotente              | 1x economia          | 1           | PASS   |
| Rollback F1-F5                 | TX limpa             | ok          | PASS   |
| Ledger único                   | 1                    | 1           | PASS   |
| History canônico / tick        | 1 / window           | ok          | PASS   |
| last 1x                        | T6                   | creditCap   | PASS   |
| history ~ economia / tick      | <= epsilon           | ok          | PASS   |
| Mismatch -> abort              | sem saldo/last       | ok          | PASS   |
| Epsilon único                  | ABS 1e-9 / REL 1e-12 | sim         | PASS   |
| Normal 8 / NFT 216             | 1 row                | testes      | PASS   |
| Catch-up estado fixo           | = sequencial         | delta~0     | PASS   |
| Catch-up estado mutável        | != replay (contrato) | documentado | PASS   |
| Redis REQUIRE fail-closed      | skip                 | ok          | PASS   |
| API/UI 1:1                     | crédito da janela    | P3 intacto  | PASS   |
| Legado >10m                    | só auditado          | SQL prod    | PASS   |
| Economia / yield / eligibility | intocados            | sim         | PASS   |

## G. Produção

DATABASE MUTATION: NÃO  
 PRODUCTION DEPLOY: NÃO  
 BALANCE MANUAL ADJUSTMENT: NÃO  
 HISTORY MANUAL ADJUSTMENT: NÃO

(Consultas RO apenas para o relatório de legado.)

## Contrato Redis (explícito)

| Config                         | Comportamento                                                 |
|--------------------------------|---------------------------------------------------------------|
| REQUIRE_REDIS_LOCK=1 sem Redis | Fail-closed — não processa                                    |
| Redis OK, lock ocupado         | skip (ok: true, sem crédito)                                  |
| Redis off + REQUIRE=0          | lock no-op; barreira = FOR UPDATE + ledger (+ UNIQUE em prod) |

## Política history

```
assert compatible (+/- epsilon) -> INSERT ... ON CONFLICT DO NOTHING  
mismatch -> ROLLBACK completo (sem saldo, sem last)
```

## Próximo passo opcional

- 5B — resegmentação controlada do legado >10 min
  - 6 — replay físico / eligibility no crédito (só com decisão arquitetural)
-